

Service Oriented Architecture REST API

ADAM BELLOUM(UVA)



Funded by
the European Union

DIGITAfrica project has received funding from the EU Horizon Europe research and innovation Programme and Switzerland under Grant Agreement No. 101187966. Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or the European Research Executive Agency (REA). Neither the European Union nor the granting authority can be held responsible for them.





Content

Service Oriented Architecture

Web services

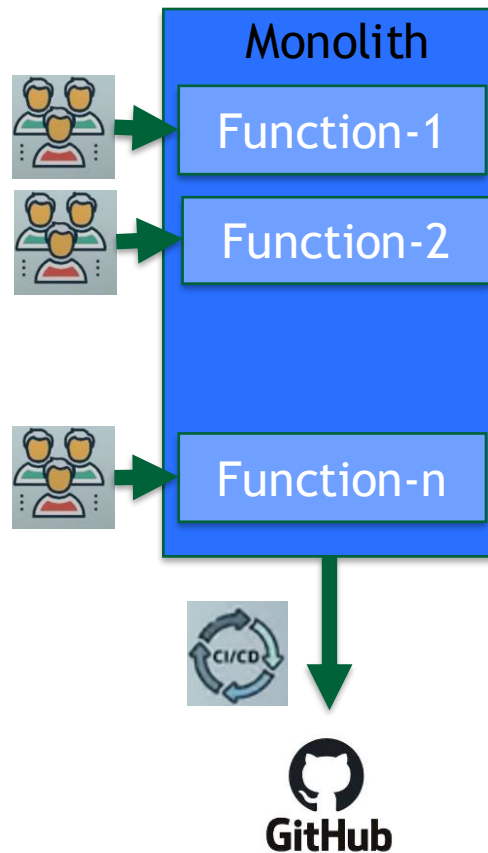
- SOAP Based Web services
- **RESTful** Web service

Important: it must be understood that *Web services* does not equal *service-oriented architecture*.





Monolith vs modular architectures



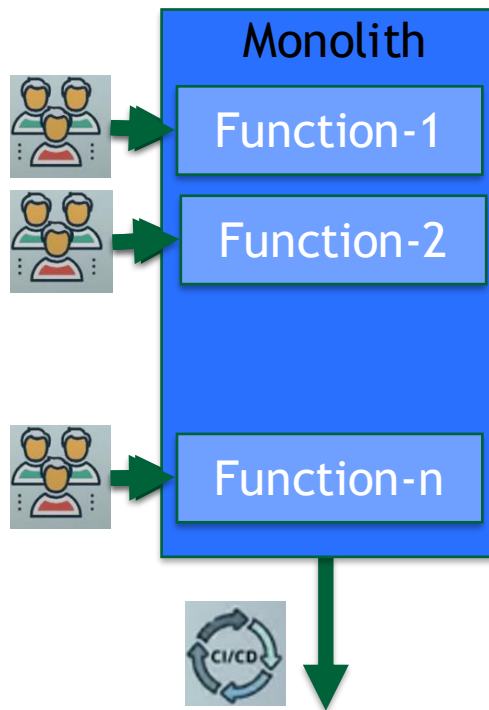
- All functions are part of single unit
- Development/ deployment are tightly coupled
- Application is written in one tech Stack
- **(-) The impact of errors is global**

With service and micro-services



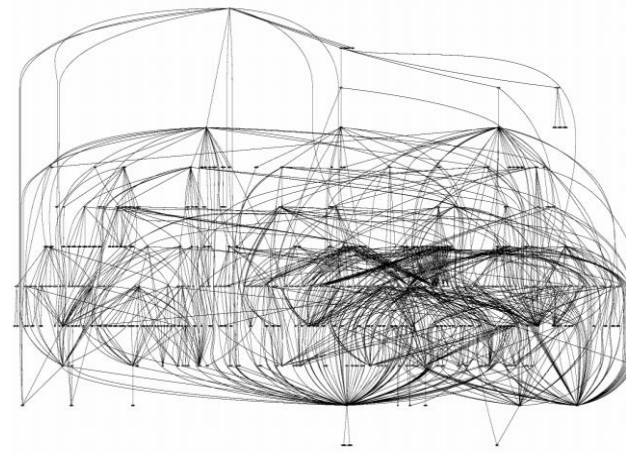
service and

Monolith vs modular architectures



Problem

Cumulative effect of decades of **growth** and evolution has produced severe **complexity**
Redundant and **non-reusable** programming
Real **integration** killer –multiplicity of interface



Callgraph of the POP

With micro-services

“It is hard to get insight into the inner workings of a Fortran application that has been worked on for **25 years!**”

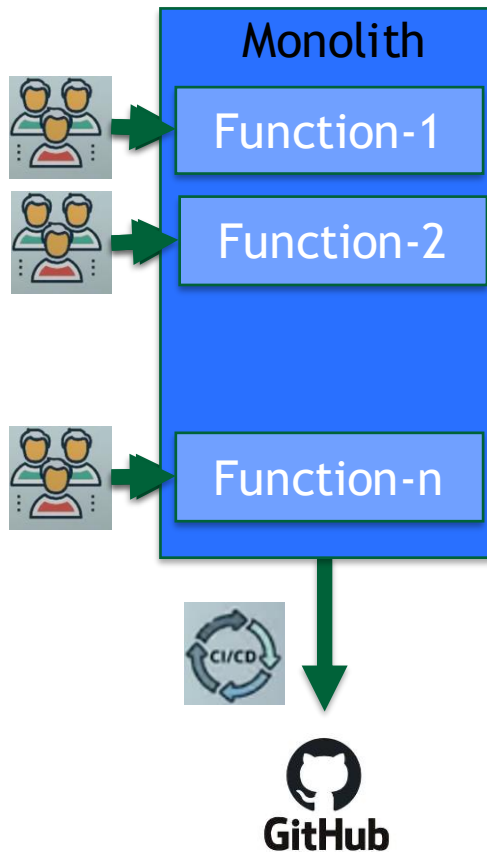
Ben van Werkhoven



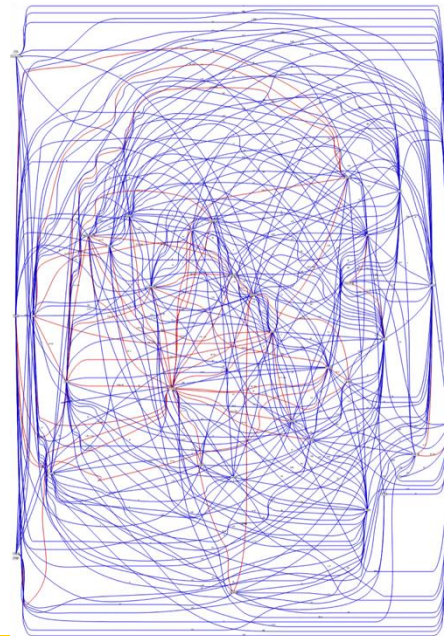
Monolith vs modular architectures

Problem

With service and micro-services



Cumulative effect of decades of **growth** and evolution has produced severe **complexity**
Redundant and **non-reusable** programming
Real **integration** killer –multiplicity of interface

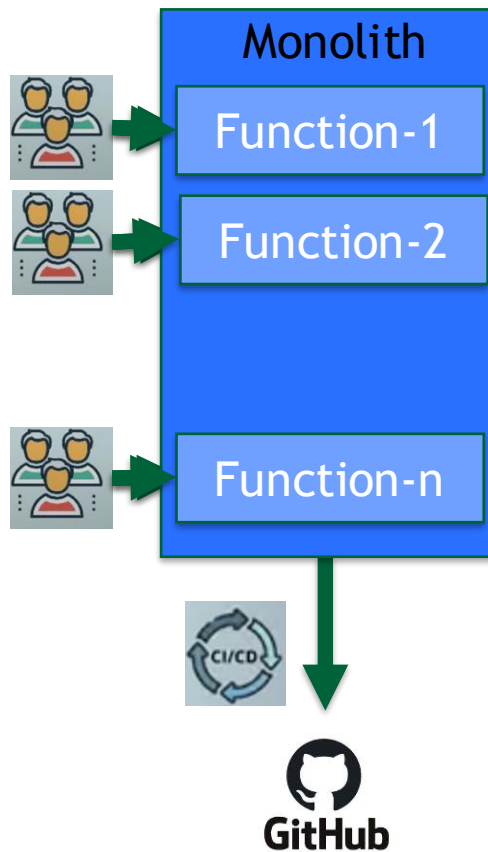


Eulerian AMR radiation-hydronamics code.

- xRage is descended from Sage, a code originally written around **1990**.
- It contains about **470K lines of source code**,
- mostly in **Fortran 90**, **C** and **C++**, and uses **MPI-only parallelism**.”



Monolith vs modular architectures



Problem

Cumulative effect of decades of **growth** and evolution has produced severe **complexity**
Redundant and **non-reusable** programming
Real **integration** killer –multiplicity of interface

“... connecting an application to data or functionality housed in another system required **complex point-to-point integration** that developers had to recreate, in part or whole, for each new development project”

SOA (Service-Oriented Architecture)

By: IBM Cloud Education⁽¹⁾ July 2019

With service and micro-services

Solution (Modular architecture)

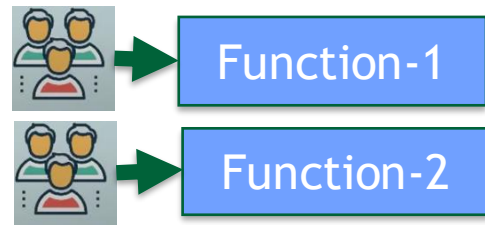


Source: figure from IBM Redbook, Patterns: Service- Oriented Architecture and Web Services

⁽¹⁾ <https://www.ibm.com/cloud/learn/soa#toc-what-is-an-sqoa7ntn>

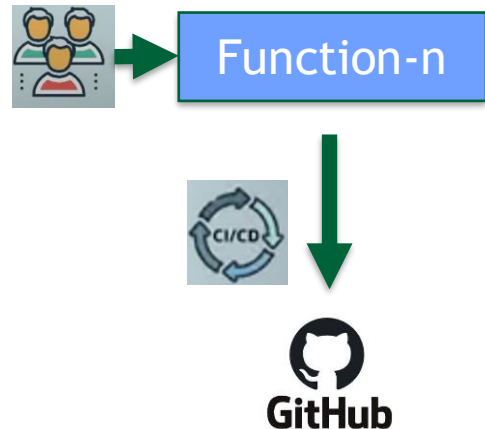


Monolith vs modular architectures



1. **How** to break down the application?
2. **How** these functions communicate?
3. **How** many functions do we need create?
4. **How** big or small the functions should (granularity) ?

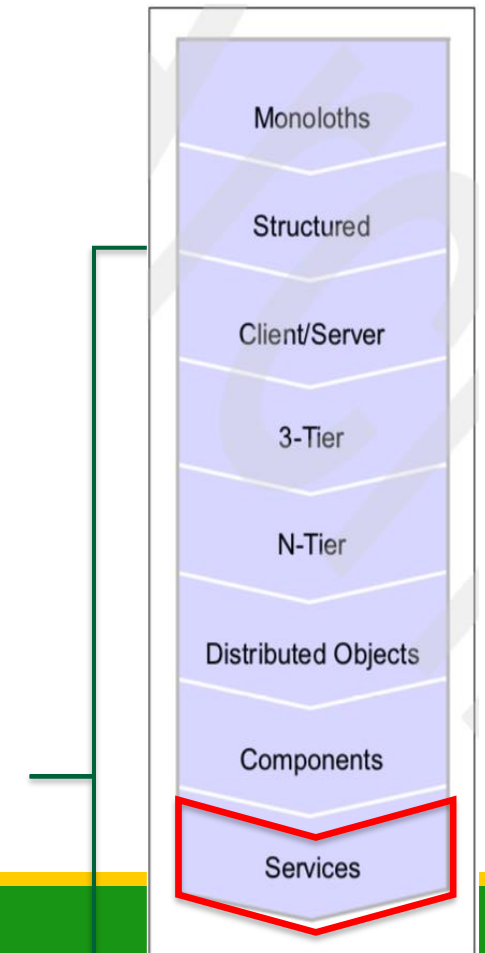
With service and micro-services



Leverage existing assets (migration of assets)
Allow for **incremental** implementations

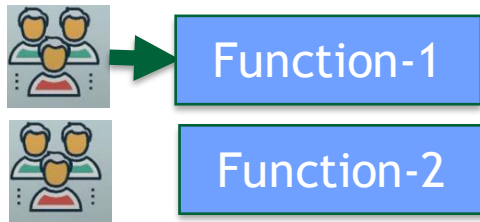
- Support **integration of new component**
 - User Interaction
 - Application Connectivity
 - Process Integration
 - Information Integration

Solution (Modular architecture)

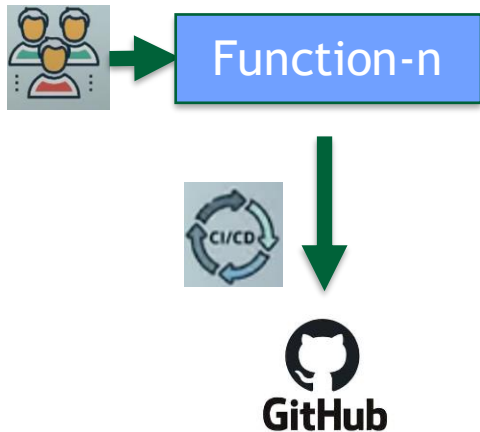




Monolith vs modular architectures

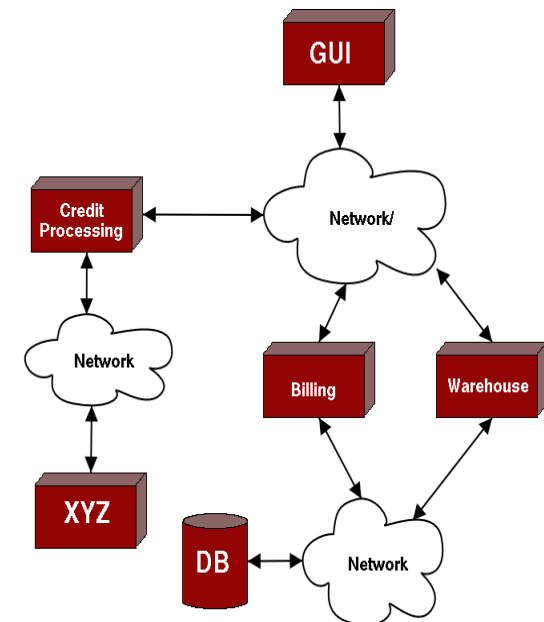


1. How to break down the application?
 - a) **Split based on Functionality not tech**
 - b) **Separation of concerns**
 - c) **Self contained and independent**



With **service** and micro-services

Service oriented Architecture





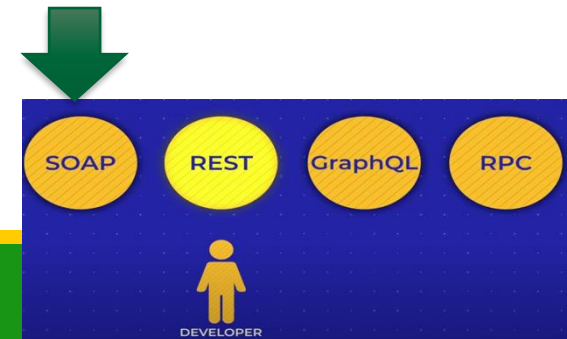
Content

Service Oriented Architecture

Web services

- SOAP Based Web services
- **RESTful** Web service

Important: it must be understood that *Web services* does not equal *service-oriented architecture*.





Web service definition

Enabling technologies

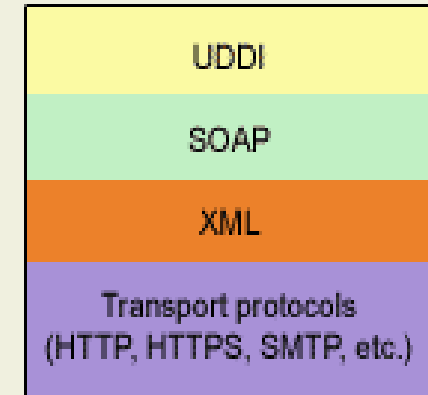
Definition [W3C]^[1]

A Web service is a software system designed to support **interoperable machine-to-machine** interaction over a network.

- It has an **interface** described in a machine-processable format (specifically WSDL).

Other systems interact with the Web service based on info in WSDL description

- SOAP messages,
- using HTTP
- with an XML serialization
- in conjunction with other Web-related standards.



Web Services Protocol Stack

Source: Java™ Web Services Unleashed

^[1] <https://www.w3c.org/TR/ws-arch/>



SOAP Web Services

Header provide information about the message

Body contains the actual data of the request

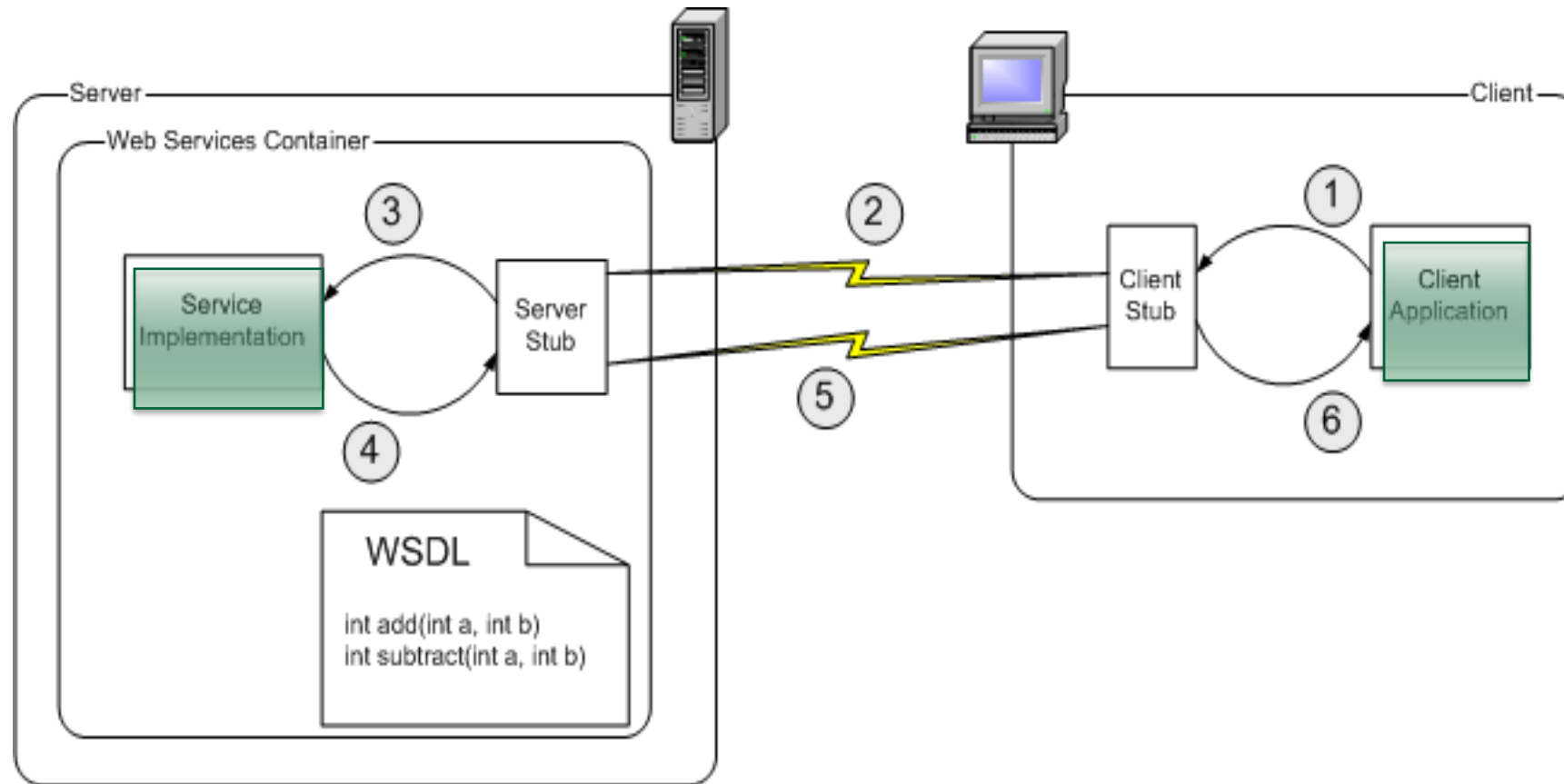
Envelope

S

```
1. <?xml veriosn="1.0" encoding ="utf-8"?>
2. <soap:envelope xmlns:soap="http://schema.xmls.soap.org/soap/envelop/
3.   xmlns:c= "http://acmeOrders.com/OrederService"
4.   xmlns:xsi="http://www.w3.org/2001/xmlSchma-onstence">
5.   <soap:Body>
6.     <c:OrderMessage>
7.       <localeElement>
8.         <FirstName>John</FirstName>
9.         <LastName>smith</LastName>
10.        <Sreet>Science Park</Street>
11.        <city>amsterdam</city>
12.        <zipcode>1098XH</zipcode>
13.        <PartNumber>ABC123</PartNumber>
14.        <PartNumber>1</PartNumber>
15.      </localeElement>
16.    </c:OrderMessage>
17.  </soap:Body>
```



What a Web Service Application Looks Like





Developing a web service

SERVICE DEVELOPER

1. Implement all the functionality of the Web Service
2. Define the interface of the Web service [WSDL]
3. Deploy the Web service in a service container

CLIENT DEVELOPER

1. Pre-requisite: URI and the WSDL of the web service
2. Invoke the operation as described in the WSDL (give the correct parameters)

Note: Integrated Development Environment (IDE) which support web service development - Visual Studio, IntelliJ, pycharm, NetBeans, eclipse, ...

Feature	Meaning in SOAP Web Se
Portability	Service can run on different environments without mo
Platform Independence	Service does not depend hardware
Language Independence	Service can be built and c in different programming languages

Web Service pros. and cons.

ADVANTAGES

Portability

- platform-independent
- language-independent

Standard technology stack

Enable to build loosely couple software architectures

DISADVANTAGES

Overhead

Lake of versatility

- State **Stateless** vs Stateful



Example

WEB SERVICE

FirstService.asmx

NET FRAMEWORK

- PROXY OF THE WEB SERVICE → WSDL
 - FirstService.cs
 - FirstService.dll (**proxy**)

```
1. <%@ WebService language = "C#" class = "FirstService" %>
2. using System;
3. using System.Web.Services;
4. using System.Xml.Serialization;
5. [WebService(Namespace = "http://localhost/MyWebServices/")]
6. [WebMethod]
7. public class FirstService : WebService {
8.     [WebMethod]
9.     public int Add(int a, int b) { return a + b; }
10.    public String SayHello() { return "Hello World"; } }
```



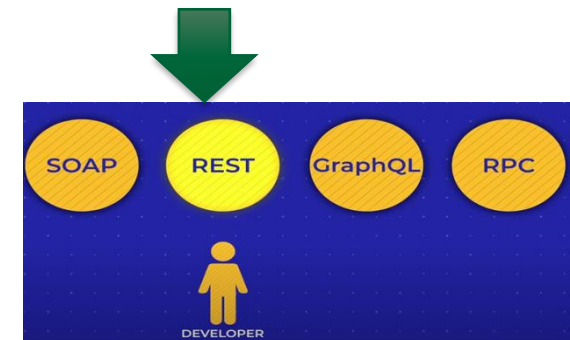

Content

Service Oriented Architecture

Web services

- SOAP Based Web services
- **RESTful** Web service

Simplicity Wins Over Complexity



[Next section](#)



What is REST

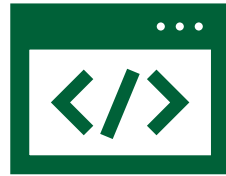
REST stands for **Re**presentational **S**tate **T**ransfer.

Introduced by Roy Fielding in 2000 in his Ph.D. dissertation to describe a **design pattern for implementing networked systems.**

“In many ways, the World Wide Web itself, based on HTTP, can be viewed as a REST-based architecture”

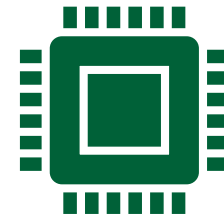


REST - Not a Standard



REST is just a design pattern

You can only understand it and design your Web services to it.



REST does prescribe the use of standards:

HTTP

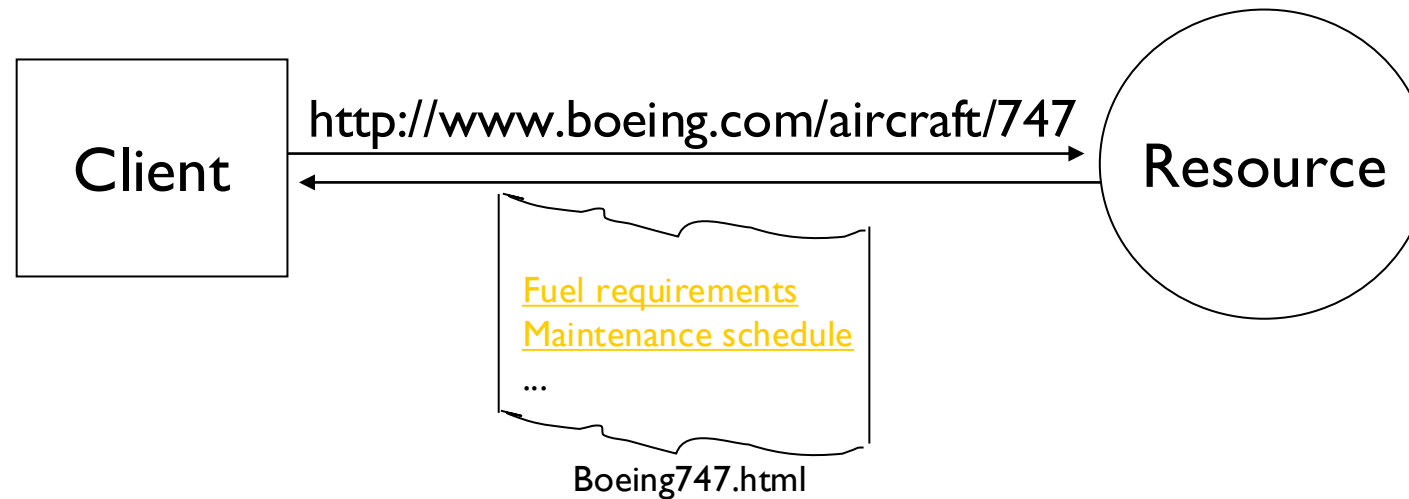
URL

XML/HTML/GIF/JPEG/*etc.* (**Resource Representations**)

text/xml, text/html, image/gif, image/jpeg, *etc.* (Resource Types, MIME Types)

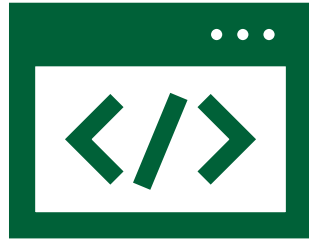


Why is it called "Representational **State** Transfer? "





Representational State Transfer



REST is intended to evoke an image of how a well-designed Web application behaves:



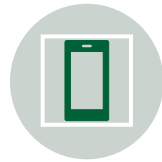
Web App is a **network of web pages** – **representing different state of the WebApp** (a virtual state-machine). **state transitions** are triggered by hyperlinks clicks

Six constraint of a RESTful Architecture

Constraints



CLIENT-
SERVER



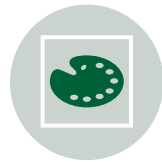
STATELESS



CACHE



UNIFORM
INTERFACE



LAYERED
SYSTEM



CODE ON
DEMAND *



Features

Scalability

Simplicity

Modifiability

Portability

Reliability

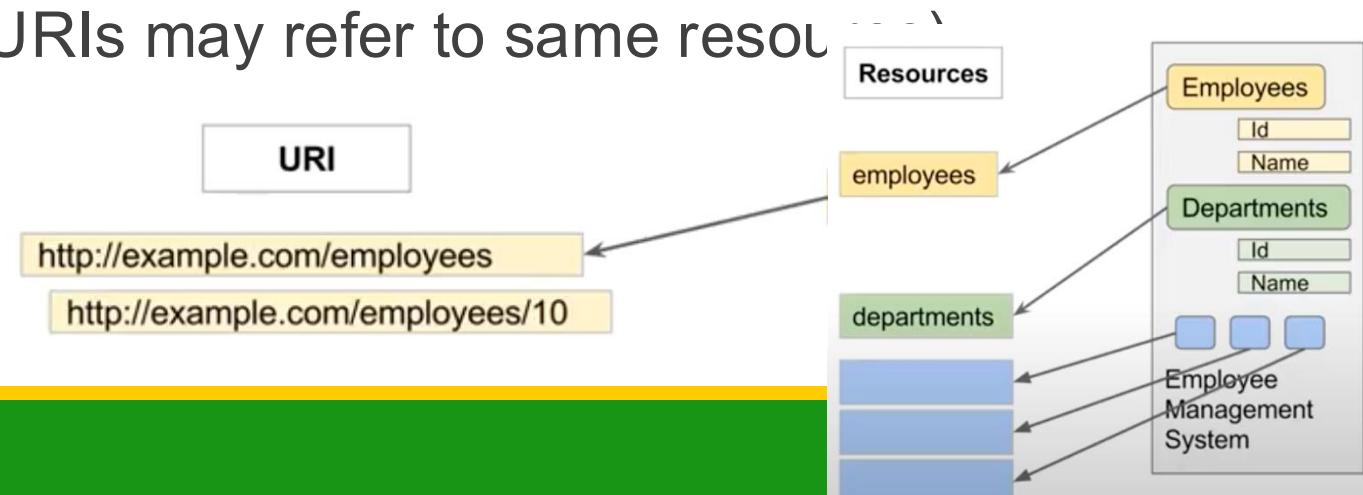
...



Resource Based

Resource is an object with

- Type,
- Associated data,
- Relationships to other resources,
- Methods that operate on it
- Identified by URIs – (Multiple URIs may refer to same resource)

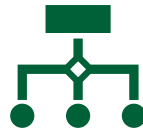




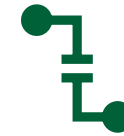
REST Fundamentals



Create a resource
for every service.



Identify each resource
using a URL.



The data that a Web
service returns
should link to other
data.

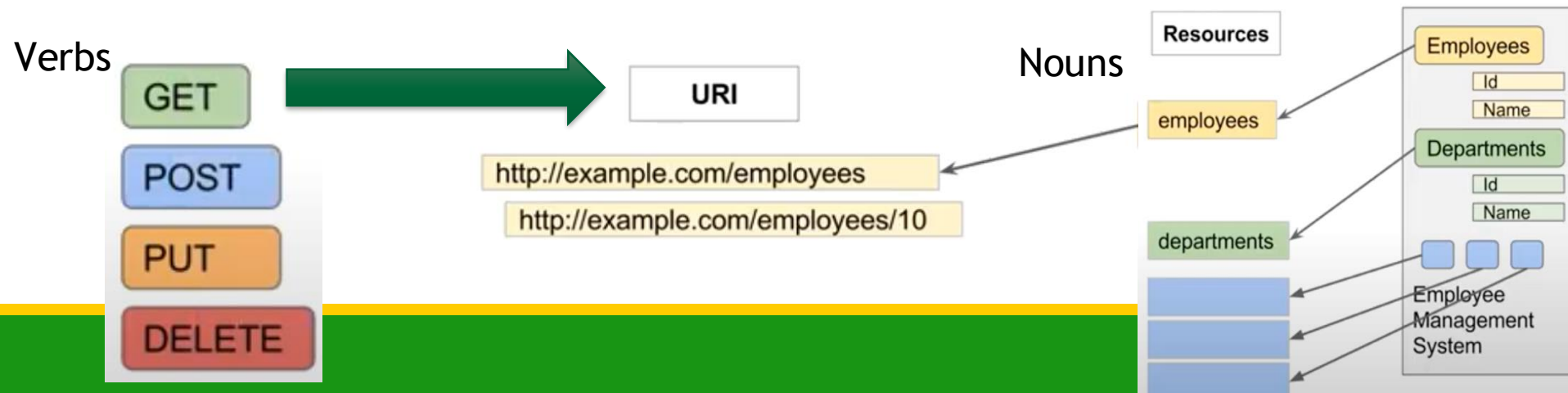
Thus, design your data as a network of information.



The REST Design Pattern

Most web interactions are done using HTTP and just four operations:

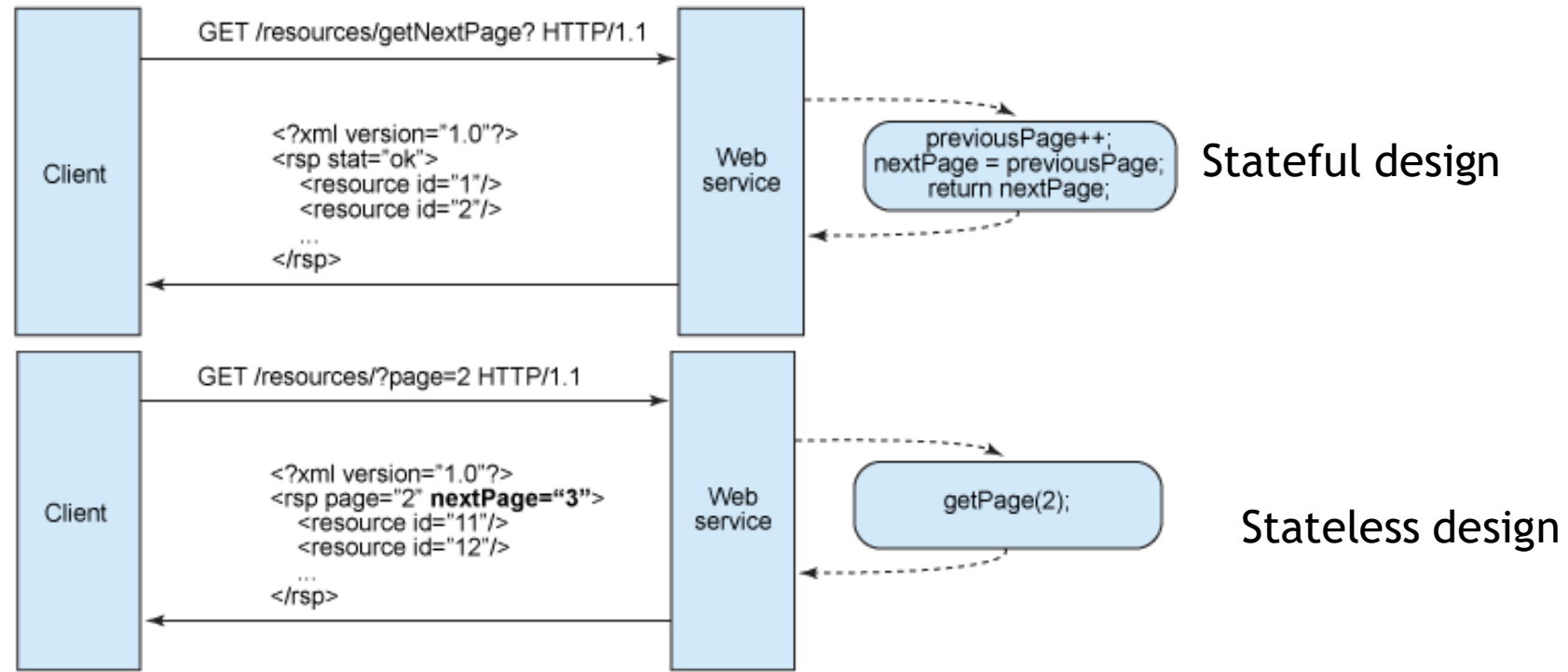
- **C**reate information (HTTP POST)
- **R**etrieve information (HTTP GET)
- **U**ppdate information (HTTP PUT)
- **D**eleate information (HTTP DELETE)





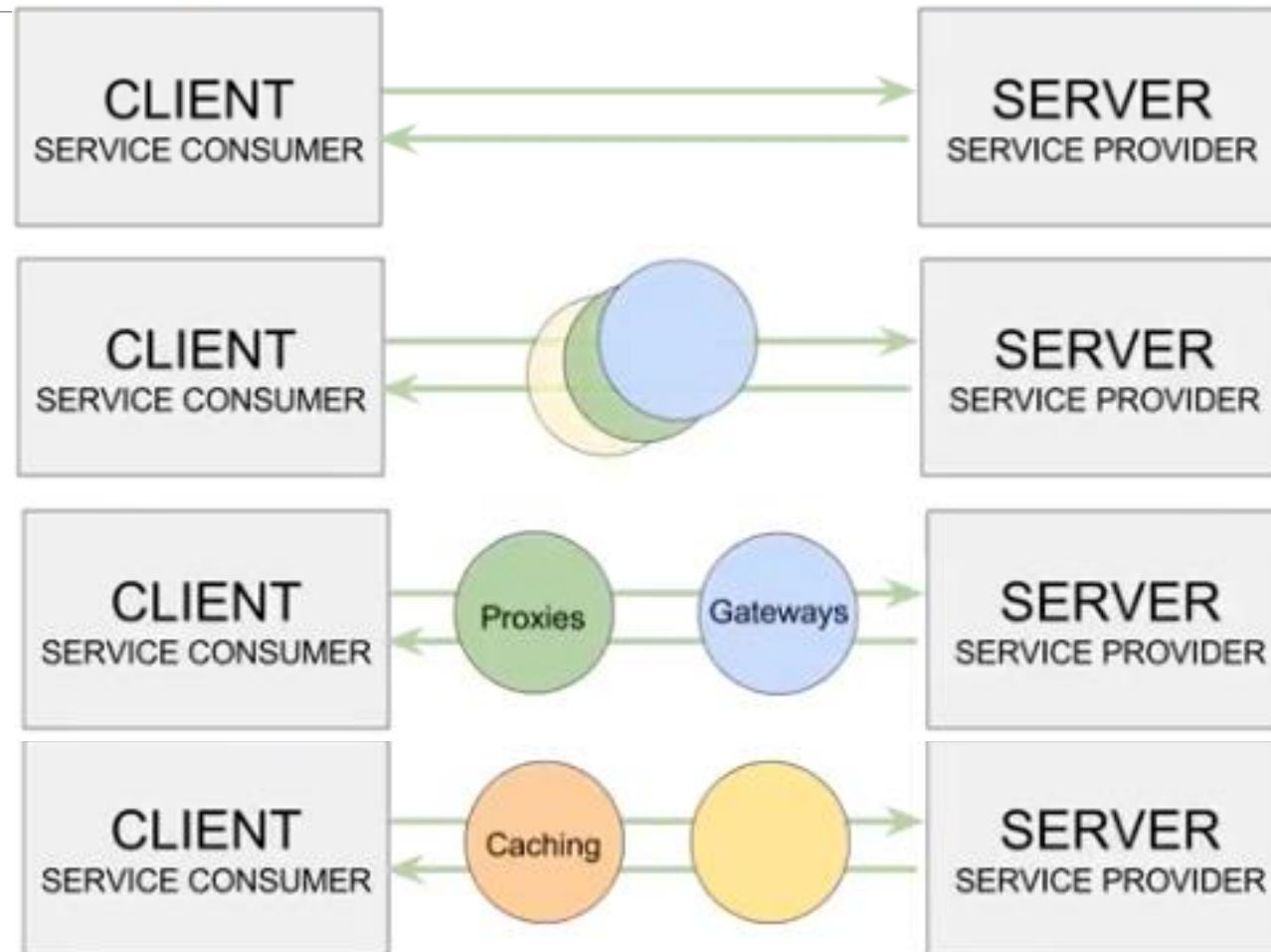
Be stateless ...

Design a service from which an application may request the **next page** in a **multipage result set**





Layered architecture ...

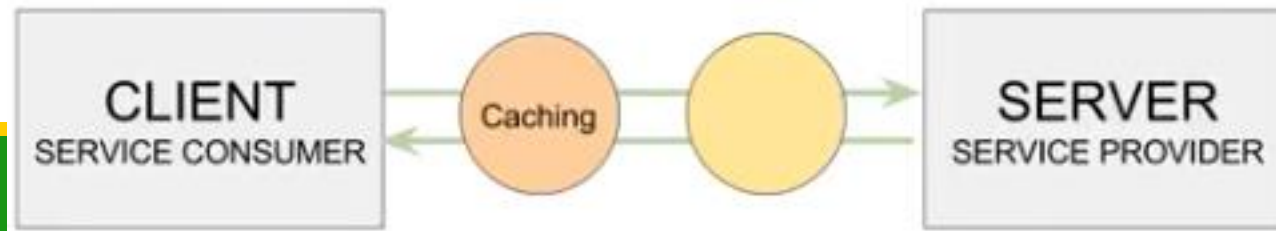




Caching ...

- Caching is done on the client side
- Example of the Response of server

URL	Status	Domain	Size
GET 2	304 Not Modified	127.0.0.1:9292	1.4 KB
Headers Response Cache HTML Cookies			
Response Headers view source			
Age 100			
Cache-Control max-age=180, public			
Connection close			
Date Wed, 26 Sep 2012 09:41:16 GMT			
Etag "2fd5257a3dd12a2b0d130931f3bc1ab5"			
Server thin 1.4.1 codename Chromeo			
X-Content-Digest 7ce9d1010d836cf77d921adc615ed387d5a59c91			
X-Rack-Cache fresh			
X-Request-Id ca28f6820243fe9fac8c7516a01c5edd			
X-Runtime 0.122089			
x-ua-compatible IE=Edge			



Expose directory structure-like URIs

Self-Documenting



Examples

How would you define URLs of a discussion threading service that gathers **topics**:

<http://www.myservice.org/discussion/topics/{topic}>

What If the resources is now organized by **date**

<http://www.myservice.org/discussion/2017/04/04/{topic}>

Feature	Concept
Self-Documenting	The URL itself tells you exactly what you are looking at.
Deriving Relations	By looking at the URI, you can guess how to find related data.
Hierarchical Flow	URIs follow a logical "Parent/Child" relationship.



Example

Create an API will take the form of a distant reading archive—a book catalog with three endpoint

- browse the book catalog welcome page
- request the list of all books
- Select a book by ID

books

```
1. books = [  
2.   { 'id': 0,  
3.     'title': 'A Fire Upon ...',  
4.     'author': 'Vernor Vinge',  
5.     'published': '1992'  
6.   },  
7.   { 'id': 1,  
8.     'title': 'The Ones Who ...',  
9.     'author': 'Ursula K. Le Guin',  
10.    'published': '1973'  
11.  },  
12.  ...  
13.]
```

distant reading archive

GET



<http://example.com/api/v1/resources/books/all>

GET



<http://example.com/api/v1/resources/books/books?id=01>

GET



<http://example.com/api/v1>

Example



Create an API will take the form of a distant reading archive—a book catalog

api.py

```
1. import flask
2.
3. app = flask.Flask(__name__)
4. app.config["DEBUG"] = True
5.
6. @app.route('/', methods=['GET'])
7. def home():
8.     return "<h1>Distant Reading Archive</h1><p>"
9. @app.route('/api/v1/resources/books/all', methods=['GET'])
10. def api_all():
11.     return jsonify(books)
12.
```

```
1. books = [
2.     { 'id': 0,
3.       'title': 'A Fire Upon ...',
4.       'author': 'Vernor Vinge',
5.       'published': '1992'
6.     },
7.     { 'id': 1,
8.       'title': 'The Ones Who ...',
9.       'author': 'Ursula K. Le Guin',
10.      'published': '1973'
11.     },
12.     ...
13.]
```

- <http://127.0.0.1:5000/>
- <http://127.0.0.1:5000/api/v1/resources/books/all>



Example

Create an API will take the form of a distant reading archive—a book catalog

api.py

13. ...

14. @app.route('/api/v1/resources/books', methods=['GET'])

15. def api_id():

16. if 'id' in request.args:

17. id = int(request.args['id'])

18. else:

19. return "Error: No id field provided. specify an id."

20. results = []

21. for book in books:

22. if book['id'] == id:

23. results.append(book)

1. <http://127.0.0.1:5000/>

2. <http://127.0.0.1:5000/api/v1/resources/books/all>

3. <http://127.0.0.1:5000/api/v1/resources/books?id=01>



Content

Service Oriented Architecture

Web services

- SOAP Based Web services
- **RESTful** Web service
- Documentation

OpenAPI : <https://swagger.io/specification/>

[Next Section](#)

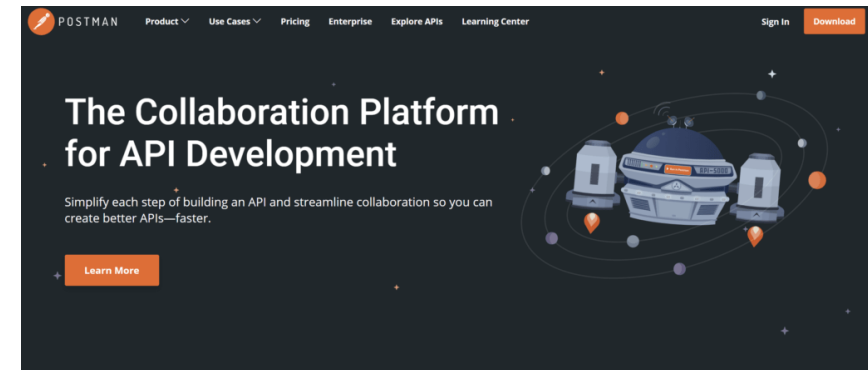


Test your REST API

use one of the existing API Client frameworks

- API Client → develop, test, share, and document APIs

Example: postman^[2] , postman Beginner Tutorial^[3]



[1] <https://www.postman.com>

[2] <https://www.youtube.com/watch?v=juldrxDrSH0>



On behalf of the DIGITAfrica consortium



Thank you!